

Package ‘Gtheory4LLM’

September 12, 2026

Type Package

Title Generalizability Theory for LLM Subjective Tasks

Version 0.0.6

Author Jin Liu [aut, cre, cph] <Veronica.Liu0206@gmail.com>

Maintainer Jin Liu <Veronica.Liu0206@gmail.com>

Description Studies the reliability and generalizability of subjective judgments produced by large language models (LLMs), including annotation, rating, and structured LLM-as-a-judge tasks. Specifies evaluator, prompt, generation, and repeated-run facets through crossed or explicitly nested random sources with configurable item interactions. Fits univariate models, joint Gaussian models, and joint discrete models for binary, ordinal, and unordered categorical outcomes, with source-specific covariance. Gaussian models use exact balanced likelihood; discrete models use a dense first-order Laplace approximation with Gaussian latent random effects. Supported balanced decision studies compare evaluator, prompt, and replication allocations using observed Gaussian or explicitly requested latent binary and ordinal reliability. Scalar nominal reliability and joint Gaussian-discrete fitting are not implemented. Includes synthetic example panels illustrating hate-speech, mental-health, and drug-review annotation tasks. Discrete fitting is limited to small models; the preflight report describes supported designs and computational limits. Generalizability coefficients follow the variance-decomposition framework of Brennan (2001) <[doi:10.1007/978-1-4757-3456-0](https://doi.org/10.1007/978-1-4757-3456-0)>.

License GPL-3

Encoding UTF-8

Depends R (>= 4.2.0)

Imports OpenMx (>= 2.22.11), graphics, stats, utils

Suggests lme4, ordinal

URL <https://github.com/Veronica0206/Gtheory4LLM>

BugReports <https://github.com/Veronica0206/Gtheory4LLM/issues>

Collate 'design.R'

'family.R'

'gaussian_engine.R'

'gaussian.R'

'discrete.R'

'preflight.R'

'fit.R'

'reliability.R'

'examples.R'

Contents

Gtheory4LLM-package	2
gt_components	3
gt_control	4
gt_design	7
gt_diagnostics	8
gt_dstudy	10
gt_example	11
gt_family	12
gt_fit	13
gt_preflight	15
gt_reliability	17
gt_score	18
gtheory_datasets	19
gtheory_drug_review	20
gtheory_hate_speech	22
gtheory_mental_health	23
print.summary.gt_fit	25
Index	27

Gtheory4LLM-package	<i>Generalizability Theory for LLM Subjective Tasks</i>
---------------------	---

Description

Studies the reliability and generalizability of subjective judgments produced by large language models (LLMs), including annotation, rating, and structured LLM-as-a-judge tasks. Supported balanced decision studies compare projected reliability across evaluator, prompt, and repeated-run allocations.

Details

Use `gt_design()` to declare random sources, `gt_preflight()` to inspect design and resource limits, `gt_family()` to distinguish the observation family, and `gt_fit()` to estimate a model. Inspect `gt_diagnostics()` before interpreting coefficients, then use `gt_dstudy()` to compare an allocation grid with a substantively chosen reliability target.

The modeling interface supports selected item interactions, crossed and explicitly nested sources, and joint multivariate source covariances. Measurement roles are specified by the research design; facet names do not automatically impose nesting or remove item interactions. Traditional method-of-moments G theory already includes random effects and crossed/nested designs. This package integrates design-specific covariance models, outcome-appropriate likelihoods, and supported decision studies; superiority in estimation accuracy requires comparative validation.

Gaussian outcomes use exact balanced likelihood. Binary, ordinal, and unordered categorical outcomes use their specified observation distributions with Gaussian latent random effects, fitted by a dense first-order Laplace approximation. Joint Gaussian models and joint discrete models are supported; a joint Gaussian-discrete likelihood is not implemented.

Observed Gaussian reliability and latent binary or ordinal reliability are distinct estimands. Scalar nominal reliability is not implemented. Decision studies require a complete balanced coded panel. Their projections depend on fitted source covariances and the specified universes of evaluators and

prompts. Adding arbitrarily chosen LLMs does not ensure those assumptions hold. Projections do not quantify sampling uncertainty, automatically find a minimum allocation, or establish annotation accuracy against human judgments.

Three synthetic example panels illustrate hate-speech, mental-health, and drug-review annotation tasks, with eight outcome sets. These are software examples, not real LLM outputs, source-study records, or clinical data. The [dataset overview](#) and dedicated [Hate-Speech](#), [Mental-Health](#), and [Drug-Review](#) topics describe variables, category mappings, generation, and the task literature. The cited studies provide background only; they supplied none of the simulated observations.

This package is distributed under GPL-3. Numerical acceptance does not establish Laplace approximation adequacy, a global optimum, or empirical validity. Bootstrap, jackknife, and general uncertainty intervals are not implemented. The installed ‘doc/LLM-workflow.R’ tutorial demonstrates fitting through reliability and a decision study using synthetic continuous judgments.

Author(s)

Jin Liu (author and maintainer), <Veronica.Liu0206@gmail.com>

References

- Brennan, R. L. (2001). *Generalizability Theory*. Springer. doi:10.1007/9781475734560.
- Jiang, Z., Raymond, M., Shi, D., and DiStefano, C. (2020). Using a linear mixed-effect model framework to estimate multivariate generalizability theory parameters in R. *Behavior Research Methods*, 52, 2383–2393. doi:10.3758/s1342802001399z.

See Also

[gt_design](#), [gt_preflight](#), [gt_fit](#), [gt_dstudy](#), [gt_example](#), [glm](#)

gt_components

Extract Source Covariances or Correlations

Description

Returns source-specific matrices on their modeled scale without transforming latent variances into observed-score variances.

Usage

```
gt_components(fit, correlation = FALSE, tolerance = 1e-8)
```

Arguments

fit	A <code>gt_fit</code> object.
correlation	Whether to return correlation matrices instead of covariances.
tolerance	Nonnegative relative variance threshold for reporting correlations. Entries involving a variance at or below this threshold times the reference variance scale are NA.

Details

Gaussian correlation checks use observed outcome variances as the reference scales. When those are unavailable, the largest variance within each source matrix is the reference. Covariances for discrete outcomes refer to identified latent predictors or category contrasts. Correlations involving negligible source variances are suppressed rather than presented as stable estimates.

Value

A named list of matrices, with scale and interpretation attributes. Correlation extraction adds the threshold and boundary policy.

See Also

[gt_fit](#), [gt_diagnostics](#)

Examples

```
d <- expand.grid(item = 1:12, rater = 1:3)
d$score <- sin(d$item) + d$rater / 5 + cos(d$item * d$rater) / 4
design <- gt_design("item", "rater", random = ~ item + rater)
fit <- gt_fit(d, "score", design,
  control = gt_control(gaussian = list(check_hessian = FALSE, retry_seed = 42)))
gt_components(fit)
```

gt_control

Configure Gaussian and Discrete Fitting and Acceptance Checks

Description

Collects backend controls in separate named lists. Backend-specific names and values are checked when a model is fitted.

Usage

```
gt_control(gaussian = list(), discrete = list())
```

Arguments

gaussian	Uniquely named list of Gaussian settings, described below.
discrete	Uniquely named list of discrete settings, described below.

Details

Gaussian controls and their defaults are:

- optimizer: CSOLNP.
- max_iterations: 3000.
- tolerance: 1e-12.
- check_hessian: TRUE.
- threads: 1.

- silent: TRUE.
- extra_tries: 9.
- retry_seed: NULL.
- start: NULL (automatic moment-based covariance starts).
- max_preparation_bytes: $512 * 1024^2$.

The allocation guard estimates preparation memory, excluding caller data and downstream OpenMx objects. An explicit retry seed preserves the caller's random-number state. Gaussian start accepts a complete named list of covariance matrices for all model sources, including Residual. Fully named matrix axes are matched independently to the outcome names; both unnamed axes use outcome order. Incomplete, duplicate, or incompatible labels are rejected. Admissible matrices receive the engine's existing interior eigenvalue floor. Diagonal models retain their repaired diagonal entries, and a pooled residual starts from the mean of those entries. These values initialize estimated parameters; they do not fix their fitted values.

Discrete resource limits are:

- max_random_dimension: 200.
- max_observations: 1200.
- max_parameters: 80.

Discrete optimizer controls are:

- maxit: 150.
- inner_maxit: 60.
- inner_tol: $1e-7$.
- reltol: $1e-7$.
- start_sd: 0.4.
- trace: 0.
- optimizer: "L-BFGS-B"; alternatively "nllminb".
- start: NULL (automatic response and covariance starts).
- covariance_parameterization: "log_cholesky".

The default covariance parameterization is unchanged. Set covariance_parameterization to "variance" to estimate nonnegative variances directly for univariate models or joint models with covariance = "diagonal". It admits an exact-zero variance. It does not implement a boundary solver for unstructured joint covariance and rejects that combination rather than removing covariances. The variance upper bound is $\exp(10)$, matching the largest diagonal variance allowed by the default log-Cholesky bounds. start_sd still denotes a starting standard deviation and must be positive.

Discrete start is a finite, uniquely named numeric vector of optimizer parameters, with complete or partial overrides of the automatic starts. Names are matched exactly; unknown names and values outside the model bounds are rejected. Automatic starts are also checked against the bounds before fitting; the optimizer never silently projects an invalid starting vector. Labels that produce ambiguous parameter names are rejected. The fitted fit\$parameters vector can initialize a refit of the same model, with unchanged family links, category coding, covariance structure, and covariance parameterization. These are optimizer coordinates: ordinal threshold parameters contain the first threshold followed by log positive gaps; log-Cholesky diagonal parameters are log standard deviations; direct-variance parameters are variances. Actual and automatic initial vectors are retained in the diagnostics, and fit\$starting_parameters records the primary start. Alternative starts

perturb fixed-effect coordinates from the primary start and use the automatic covariance starts with distinct deterministic scale perturbations. If clipping would duplicate an earlier start, a bounded fixed-effect perturbation makes the additional trial distinct.

The selected discrete optimizer remains fixed for primary fits, alternatives, and tight refinements; the engine never silently switches algorithms. `maxit` limits iterations per optimization. For `nlminb`, the function evaluation budget is $\max(200, 10 * \text{maxit})$, and `reltol` supplies its relative objective tolerance. Each optimizer uses its own native stopping rules; the same independent numerical-acceptance checks apply afterward. Optimizer choice is not evidence of approximation accuracy.

Zero is a natural lower variance boundary in this opt-in mode, not an artificial optimizer floor. Within `gt_diagnostics(fit)$diagnostics`, the `zero_variance_parameters` field lists exact zeros. These estimates must still pass independent covariance-scale stationarity and restart checks. All upper bounds and fixed-effect/threshold bounds remain artificial and still cause rejection on contact. This mode does not establish approximation accuracy or sampling properties for boundary estimates.

Discrete numerical-acceptance controls are:

- `stationarity_tol`: 1e-3.
- `validation_reltol`: 1e-10.
- `validation_inner_tol`: 1e-9.
- `alternative_starts`: 1.
- `stability_objective_tol`: 1e-6.
- `stability_parameter_tol`: 0.02.
- `bound_tol`: 1e-4.

Setting alternative starts to zero disables numerical acceptance, not just an optional report. The discrete optimization-attempt budget is $2 * (1 + \text{alternative_starts})$: one preliminary and one tight fit for the primary start and each alternative. Final-mode and stationarity evaluations are recorded separately and are not additional optimization trials. Gaussian `extra_tries` and discrete `alternative_starts` therefore have different meanings. Covariance derivative accuracy uses at most six step halvings when needed, without changing the stationarity or derivative-disagreement tolerances. Every stationarity evaluation, including the center and all fixed-effect and covariance perturbations, must have a valid, converged conditional mode and usable finite likelihood. A failed evaluation ends the diagnostic and records a validation error; an optimizer penalty is never treated as a derivative.

For a known-covariance calculation, `fixed_covariance` can provide one positive-semidefinite matrix for every discrete random source. Fully named matrices are matched and reordered to the latent dimensions; unnamed matrices are positional. Partial or incompatible dimension names are rejected. These matrices are fixed assumptions rather than estimated source covariances. The covariance parameterization setting is not used when all covariances are fixed.

The `attempts` entry of `gt_diagnostics(fit)$diagnostics` retains starts, controls, optimizer codes, messages, errors, warnings, elapsed time, and returned parameters. A computational failure during refinement, a restart, or stationarity checking prevents numerical acceptance but preserves an otherwise usable primary or alternative fit for inspection. Unusable optimizer returns are retained unchanged as `raw_result`, separately from the computational placeholder. The selected attempt, optimizer, trial count, and budget are also reported. If no fitted candidate supplies a usable conditional mode, a `gt_discrete_numerical_failure` error condition retains the attempt records instead. Invalid input still fails before fitting. Warnings captured during numerical attempts are retained in these diagnostics; user interrupts are not swallowed.

Optimizer completion, numerical acceptance, and approximation adequacy are separate. Tighter numerical tolerances do not establish the adequacy of a first-order Laplace approximation.

Value

A `gt_control` object with separate Gaussian and discrete control lists.

See Also

[gt_diagnostics](#), [gt_fit](#)

Examples

```
gt_control(gaussian = list(retry_seed = 123, check_hessian = FALSE),
  discrete = list(max_random_dimension = 100, alternative_starts = 1))
# Opt-in boundary-capable coordinates; this does not change the default.
gt_control(discrete = list(covariance_parameterization = "variance"))
gt_control(discrete = list(optimizer = "nlnmb", alternative_starts = 2))
```

gt_design	<i>Declare a Generalizability-Theory Random-Source Design</i>
-----------	---

Description

Declares an object of measurement, any number of instrumentation facets, selected object interactions, and explicit parent-scoped nesting. This constructor preserves the requested source terms; fitting validates the observed data and resolves any residual alias.

Usage

```
gt_design(object, facets, crossed = facets, nested = NULL,
  item_interactions = "complete", instrument_interactions = "complete",
  item_nested = character(), random = NULL, replicates = 1L,
  max_terms = 4096L)
```

Arguments

object	One column name for the object of measurement.
facets	Unique instrumentation column names, excluding the object.
crossed	A nonempty subset of facets allowed to form direct object interactions. Defaults to all facets.
nested	A named list giving the instrumentation parents of every non-crossed facet. Multiple parents define their joint group. Ancestors are expanded recursively; cycles are rejected.
item_interactions	"complete", "additive", or the maximum number of crossed instrumentation facets in an object interaction. The object itself is excluded from this order. Additive adds individual object-by-crossed-facet interactions.
instrument_interactions	"complete", "additive", or a maximum interaction order among crossed instrumentation facets. Declared nested groups remain present.
item_nested	Nested child names whose complete ancestor-expanded group should also interact with the object. This does not implicitly add other ancestor-by-object interactions.

random	Optional explicit one-sided formula or character vector of source terms. Formula syntax is limited to names, +, :, *, parentheses, and 0/1. Cannot be combined with explicitly supplied automatic selectors.
replicates	Positive integer observations per complete object-by-facets cell. Observed replication is checked during fitting.
max_terms	Positive integer bound on source expansion, checked before expansion.

Details

The object main effect is always present. With all facets crossed, complete object and instrumentation interactions retain the full hierarchy. At one observation per cell, a Gaussian full-cell source is indistinguishable from a free residual and is combined with it during fitting. A requested observation-level source is rejected for discrete families; users must explicitly declare a reduced model when appropriate.

Nesting defines parent-scoped groups shared across objects. It does not establish that a physical sampling design is balanced or identifiable. The exact Gaussian backend currently requires a complete coded Cartesian panel with one observation per cell. Arbitrary design declaration does not imply that every backend supports every resulting model.

Value

A `gt_design` list containing the object, facets, requested terms and their members, nesting metadata, and pending data-validation information.

See Also

[gt_fit](#), [gt_family](#)

Examples

```
complete <- gt_design("item", c("evaluator", "prompt", "temp", "seed"))
length(complete$terms_requested)
selected <- gt_design("item", c("evaluator", "prompt", "temp", "seed"),
  crossed = c("evaluator", "prompt"),
  nested = list(temp = c("evaluator", "prompt"), seed = "temp"))
selected$terms_requested
reduced <- gt_design("item", c("rater", "occasion"), random = ~ item + rater)
reduced$terms_requested
```

gt_diagnostics

Inspect Numerical Acceptance and Backend Diagnostics

Description

Reports optimizer completion separately from numerical acceptance and likelihood-approximation adequacy.

Usage

```
gt_diagnostics(fit)
```

Arguments

`fit` A `gt_fit` object.

Details

Gaussian diagnostics include independent likelihood reconstruction, covariance-scale stationarity, boundary components, and optional Hessian diagnostics. Discrete diagnostics include covariance- and parameter-scale stationarity, bound locations, and tighter-tolerance alternative-start stability. Passing these checks does not prove structural identification, a global optimum, or adequate approximation in sparse or strongly dependent data.

`selected_attempt` identifies the returned candidate using the engine's recorded trial identifier. It is NA when the Gaussian native transcript does not uniquely identify that trial. `acceptance_failures` gives reasons for rejecting the returned fit; `attempt_failures` is a two-column data frame of recorded attempt identifiers and error or rejection reasons, including earlier attempts. An empty attempt-failure table does not certify every trial: some native candidates have no independent acceptance assessment.

`boundary_sources` describes natural zero or nearly singular covariance components and does not by itself imply rejection. `parameter_bounds` lists artificial discrete-parameter bound contacts, which do prevent numerical acceptance. These shared fields summarize existing engine decisions without repeating or changing numerical checks. `summary(fit)` prints a concise version while retaining full diagnostics for programmatic inspection.

Value

A list with estimator, engine, optimizer and numerical acceptance status, approximation status, engine-specific diagnostics, resolved aliases, and data-validation notes. Shared diagnostic fields include:

- `optimizer`: the selected optimization engine.
- `optimization_trials`: the recorded trial count.
- `selected_attempt`: the returned candidate's identifier.
- `acceptance_failures`: reasons for rejecting the returned fit.
- `attempt_failures`: recorded trial errors or rejections.
- `boundary_sources`: zero or nearly singular covariance sources.
- `parameter_bounds`: artificial parameter-bound contacts.

See Also

[gt_control](#), [gt_fit](#)

Examples

```
d <- expand.grid(item = 1:12, rater = 1:3)
d$score <- sin(d$item) + d$rater / 5 + cos(d$item * d$rater) / 4
design <- gt_design("item", "rater", random = ~ item + rater)
fit <- gt_fit(d, "score", design,
  control = gt_control(gaussian = list(check_hessian = FALSE, retry_seed = 42)))
gt_diagnostics(fit)$numerically_accepted
```

gt_dstudy

*Evaluate Balanced Measurement Allocations***Description**

Computes supported G and Phi coefficients over a grid of instrumentation counts, validating the fitted context once.

Usage

```
gt_dstudy(fit, grid, scale = NULL, score = NULL)
## S3 method for class 'gt_dstudy'
plot(x, coefficient = "Erho2", ...)
```

Arguments

fit	A numerically accepted gt_fit with a complete balanced coded panel.
grid	A nonempty data frame of positive integer facet counts. Columns name declared facets; omitted facets retain their fitted counts.
scale	"observed" for Gaussian outcomes or explicitly "latent" for binary/ordinal outcomes.
score	Optional composite weights created by gt_score().
x	A gt_dstudy object for plotting.
coefficient	Plot "Erho2" (default) or "Phi".
...	Additional arguments passed to graphics::plot().

Details

These are conditional design projections using fitted source covariances. They do not refit the model, represent sampling uncertainty, or establish that larger allocations preserve the same source distributions. Restrictions and coefficient definitions are those of gt_reliability().

Value

A gt_dstudy list with completed allocation rows, coefficient results, measurements per object, scale, extrapolation flags, and fitting diagnostics. The plot method returns the object invisibly.

See Also

[gt_reliability](#), [gt_score](#)

Examples

```
d <- expand.grid(item = 1:12, rater = 1:3)
d$score <- sin(d$item) + d$rater / 5 + cos(d$item * d$rater) / 4
design <- gt_design("item", "rater", random = ~ item + rater)
fit <- gt_fit(d, "score", design,
  control = gt_control(gaussian = list(check_hessian = FALSE, retry_seed = 42)))
study <- gt_dstudy(fit, data.frame(rater = c(2, 3, 6)))
study$results
plot(study)
```

gt_example

*Load Synthetic Examples or Explicit Local Modeling Tables***Description**

Lists eight documented outcome sets or loads their design and outcome columns. Bundled observations are entirely synthetic and contain no actual LLM outputs, source-study records, raw texts, or clinical data.

Usage

```
gt_example(name = NULL, coding = c("native", "manuscript"),
  directory = NULL)
```

Arguments

name	Outcome-set name, or NULL to list available sets.
coding	"native": preserves the declared response types. "manuscript": the legacy numeric-score compatibility option for seven sets; it does not load or reproduce manuscript observations or results.
directory	Optional directory containing example RDS resources or compatible source CSV files, supplied explicitly as one nonempty path. A file named <name>.rds takes precedence over CSV loading. When omitted, the installed package loads bundled synthetic resources regardless of workspace variables. User-supplied files retain the user's own provenance.

Details

Each bundled set has 576 rows: 24 hypothetical items by three generic evaluators, two generic prompts, two temperature-setting identifiers, and two seed identifiers. All labels are generated by deterministic integer arithmetic. No raw texts, real model identities, observed prevalence, source-study annotations, or fitted parameters supply the simulated values.

The available sets are:

- hate_speech: three ordered categories.
- mental_health_7L: seven-code Gaussian working score.
- mental_health_3L: three-code Gaussian working score.
- mental_health_6flag: six binary flags.
- mental_health_3group: three binary OR groups.
- drug_review: five-category ordinal holistic sentiment.
- drug_review_4aspect: four ordinal aspects.
- mental_health_nominal: seven unordered categories.

The nominal set has native coding only. The Mental-Health 7L/3L mappings are Gaussian working-score illustrations under either option; they are not validated clinical severity scales. Background annotation papers and corpus references appear in the three task topics. They motivate the task types but are not sources for the synthetic observations.

For standalone functions, a binding named `.gt_example_data_dir` in the same enclosing environment can configure the resource directory. The source loader uses this directory in the checkout:

inst/extdata

Parent environments and callers are not searched. This standalone convention has no effect on the installed package. An invalid explicit directory raises an error instead of falling back silently.

Explicit CSV loading accepts these compatible filenames:

- 'hate_labeling_final.csv'
- 'mh_labeling_final.csv'
- 'drug_labeling_final.csv'

This compatibility interface does not make any such source file part of the bundled examples. RDS files carry their own provenance. Use [gt_preflight](#) to inspect fitting restrictions; successful loading alone is not a numerical or statistical validation.

Value

A catalog data frame when name is NULL. Otherwise a list containing data, outcome names, observation families, design roles, example name, coding, resource path, and interpretation notes. Bundled resources also include source_provenance with data_kind equal to "synthetic", author, license, and public generator identity.

See Also

[gtheory_datasets](#), [gtheory_hate_speech](#), [gtheory_mental_health](#), [gtheory_drug_review](#), [gt_family](#), [gt_fit](#)

Examples

```
gt_example()
x <- gt_example("hate_speech")
nrow(x$data)
is.ordered(x$data$score)
x$source_provenance$data_kind
nominal <- gt_example("mental_health_nominal")
levels(nominal$data$label)
```

gt_family	<i>Specify a Gaussian, Binary, Ordinal, or Unordered Categorical Outcome</i>
-----------	--

Description

Keeps observation families distinct and defines their links and category interpretation.

Usage

```
gt_family(family = "gaussian", link = NULL, levels = NULL,
  reference = NULL)
```

Arguments

family	One of "gaussian", "binary", "ordinal", or "categorical".
link	Gaussian uses identity; binary and ordinal use probit (default) or logit; unordered categorical uses softmax.
levels	Distinct category labels. Binary levels are negative then positive. Ordinal levels give the substantive order. Categorical levels identify categories without assigning an order. Gaussian outcomes have no category levels.
reference	One unordered categorical reference category. Defaults during fitting to the first declared category.

Details

Numeric binary responses must be 0/1. Ordinal and categorical families require at least three categories; use the separate binary family for two categories. Ordinal order must come from declared levels or an ordered factor. Every declared category must occur in the fitted data. Numeric Gaussian outcomes are never constructed by silently converting factors.

Categorical random effects use reference-category contrasts. An unstructured covariance can be transformed under a reference change; a diagonal contrast covariance imposes a model restriction that depends on the reference category.

Value

A `gt_family` object describing the observation distribution.

See Also

[gt_fit](#)

Examples

```
gt_family()
gt_family("binary", link = "logit")
gt_family("ordinal", levels = c("low", "middle", "high"))
gt_family("categorical", levels = c("red", "blue", "green"), reference = "blue")
```

gt_fit

Fit Univariate or Joint Multivariate Generalizability Models

Description

Fits selected random-source covariance models using an exact balanced Gaussian likelihood or a dense first-order Laplace likelihood for discrete outcomes.

Usage

```
gt_fit(data, outcomes, design, family = gt_family("gaussian"),
  estimator = NULL, covariance = "unstructured", residual = NULL,
  control = gt_control())
## S3 method for class 'gt_fit'
print(x, ...)
## S3 method for class 'gt_fit'
summary(object, ...)
```

Arguments

data	Data frame containing the object, all declared instrumentation columns, and outcomes. Missing values are not silently dropped.
outcomes	One or more unique outcome column names.
design	A declaration created by <code>gt_design()</code> .
family	One <code>gt_family</code> object for all outcomes, or a named list matching outcomes exactly. Multiple discrete families can be modeled jointly. Joint Gaussian-discrete likelihoods are not implemented.
estimator	Gaussian: "REML" (default) or "ML". Discrete: "ML_Laplace".
covariance	"unstructured" or "diagonal"; Gaussian also accepts named source overrides, with unspecified sources using diagonal covariance. Discrete source covariance uses the common requested structure.
residual	The Gaussian residual structure. Choose "unstructured", "diagonal", or "pooled"; pooled gives equal outcome variances. Omit for discrete outcomes.
control	A <code>gt_control</code> object.
x, object	A fitted <code>gt_fit</code> object for the print or summary method.
...	Further method arguments; currently unused.

Details

The Gaussian backend requires a complete coded Cartesian panel with one observation per cell and at most 4096 factorial strata. Implicit normalized Helmert contrasts avoid quadratic allocation along a large object axis. Gaussian ML profiles one fixed intercept per outcome; REML includes the unscaled-intercept constant $D * \log(N)$. Gaussian ML AIC counts covariance parameters and profiled means. Its generic BIC uses N response vectors; `ml_BIC_scalar_scores` explicitly provides the alternative $N * D$ convention. `legacy_AIC` and `legacy_BIC` preserve the archive's variance-only parameter convention. Generic Gaussian REML AIC/BIC are unavailable; explicitly named restricted-likelihood criteria are retained.

Discrete models use binary links, ordinal thresholds, or unordered reference-category softmax contrasts. They do not estimate a free Gaussian observation residual. Probit latent residual variance is identified as 1; logit as $\pi^2 / 3$. Unsupported observation-level sources, aliased free source kernels, missing categories, and resource-limit violations are rejected.

Returned binary probabilities evaluate each tail directly, preserving small representable probabilities and the declared category order. These are conditional probabilities at the random-effect mode, not population-marginal probabilities.

Numerical acceptance includes optimizer completion and independent stationarity, bound, and tighter-tolerance restart checks. First-order stationarity does not establish a global optimum. A numerically accepted Laplace fit still requires an assessment of approximation adequacy for the intended analysis. Reliability and decision studies reject fits that fail numerical acceptance.

Value

A `gt_fit` object containing source covariance matrices, estimates, likelihood and parameter counts, the resolved design and observation families, modeled data, and numerical diagnostics. Gaussian objects also retain the OpenMx model and prepared sufficient statistics. Discrete objects include thresholds or category contrasts and conditional probabilities evaluated at the random-effect mode. `print()` returns the fit invisibly. `summary()` returns a `summary.gt_fit` list with estimates, a source-variance table, full covariance components, and diagnostics. Its [print method](#) displays a concise report of numerical acceptance, selected attempts, boundaries, and estimates.

See Also

[gt_design](#), [gt_control](#), [gt_diagnostics](#)

Examples

```
d <- expand.grid(item = 1:12, rater = 1:3)
d$score <- sin(d$item) + d$rater / 5 + cos(d$item * d$rater) / 4
design <- gt_design("item", "rater", random = ~ item + rater)
fit <- gt_fit(d, "score", design,
  control = gt_control(gaussian = list(check_hessian = FALSE, retry_seed = 42)))
print(fit)
summary(fit)$minus2loglik
```

gt_preflight	<i>Inspect Data, Source Dimensions, and Backend Limits Before Fitting</i>
--------------	---

Description

Reports observed design counts, random-source dimensions, parameter counts, and the current backend's structural and resource restrictions without optimizing a model. Uses the same category and source-resolution rules as fitting.

Usage

```
gt_preflight(data, outcomes, design, family = gt_family("gaussian"),
  covariance = "unstructured", residual = NULL,
  control = gt_control())
```

```
## S3 method for class 'gt_preflight'
print(x, ...)
```

Arguments

data	A nonempty data frame with unique column names.
outcomes	One or more outcome column names.
design	A design from gt_design .
family	A family from gt_family , or a named family list matching the outcomes.
covariance	The source covariance specification accepted by gt_fit .
residual	Gaussian residual covariance specification, or NULL for the default. Do not specify this argument for discrete outcomes.
control	Engine controls from gt_control . Resource limits are read from the selected engine's controls.
x	An object returned by gt_preflight.
...	Reserved for print-method compatibility.

Details

Each random source has one predictor dimension per Gaussian, binary, or ordinal outcome and one dimension per nonreference category for a categorical outcome. Its random dimension is the product of that dimension count and its observed grouping count. These are conceptual random-effect dimensions for Gaussian fitting; the exact Gaussian backend instead uses factorial contrasts.

The source table counts free variance/covariance parameters under the requested covariance structure. Fixed discrete source covariance matrices contribute zero free covariance parameters. Gaussian observation parameters are the profiled outcome means; discrete observation parameters are intercepts or ordinal thresholds. The total includes these observation parameters and the Gaussian residual covariance parameters. It is not a residual degrees-of-freedom count or an information-criterion definition.

Gaussian checks require a complete coded Cartesian panel with one measurement per cell and resolve a full-cell source into the residual when appropriate. Discrete fitting can admit missing whole cells, but analytic reliability and D studies still require a complete balanced coded panel. Within configured size limits, preflight also applies the current discrete source-kernel checks. When an earlier structural or resource check fails, `kernel_check` explicitly reports that the kernel checks were skipped. Source counts then refer to the requested model if family-specific resolution failed.

The Gaussian allocation estimate covers preparation arrays only. The discrete byte count covers one dense random-design matrix and one random-effect Hessian only. Neither predicts peak memory or runtime. Fewer than five observed groups triggers a descriptive note, not a rejection rule. Starting values and optimizer-specific controls are validated by fitting.

Passing preflight does not establish statistical identification, reliable estimation, numerical acceptance, or Laplace-approximation adequacy. Supported scales remain conditional on a numerically accepted fit: observed Gaussian scores, or explicitly requested latent binary/ordinal responses. Latent coefficients describe averages of latent responses, not majority-vote decisions or observed label proportions. No scalar nominal coefficient is implemented.

Value

A `gt_preflight` list with observed counts and replication, source and parameter totals, resource estimates, and interpretation notes. It also contains:

- `sources`: a data frame describing the random sources.
- `checks`: a data frame of the structural and resource checks.
- `fitting_feasible`: whether those checks passed.
- `supported_reliability_scales`: conditional coefficient scales.

Feasibility is not a fitting or identification result. Invalid input types or observation categories raise an error. Unsupported observed designs and exceeded resource limits are reported as failed checks.

See Also

[gt_fit](#), [gt_diagnostics](#), [gt_reliability](#), [gt_dstudy](#)

Examples

```
# Entirely synthetic evaluator/prompt data; no API or study data are used.
d <- expand.grid(item = 1:8, evaluator = 1:3, prompt = 1:2)
d$score <- sin(d$item) + d$evaluator / 5 + cos(d$item * d$prompt) / 4
design <- gt_design("item", c("evaluator", "prompt"),
  random = ~ item + evaluator + prompt + item:evaluator)
report <- gt_preflight(d, "score", design)
```

```

print(report)
report$sources

# Missing whole cells block Gaussian fitting and analytic reliability.
incomplete <- gt_preflight(d[-1, ], "score", design)
incomplete$checks[!incomplete$checks$passed, ]

```

gt_reliability

*Compute Balanced G and Phi Coefficients***Description**

Forms universe-score, relative-error, and absolute-error covariance matrices under a declared balanced averaging design.

Usage

```
gt_reliability(fit, scale = NULL, score = NULL, design = NULL)
```

Arguments

fit	A numerically accepted gt_fit with a complete balanced coded panel.
scale	"observed" for Gaussian outcomes (default); explicitly request "latent" for binary/ordinal outcomes.
score	Optional gt_score weights. Per-outcome results are always returned.
design	Optional named positive integer instrumentation counts overriding the fitted allocation. Unspecified counts retain their fitted values.

Details

The object main-effect covariance defines universe-score variation. Every object-containing error component contributes to relative error, and every non-object-main component contributes to absolute error. Each component is divided by the product of the instrumentation counts in its grouping term; the residual is also divided by declared cell replication.

Coefficients for discrete outcomes are identified latent-response coefficients. They describe the average latent response under the declared measurement design, not binary proportions, majority-vote labels, or observed ordinal scores. Scalar reliability for unordered categories, observed discrete reliability, and unbalanced decision-study integration are not implemented. Instrumentation facets are averaged as random sources; the function does not infer an arbitrary fixed-facet score universe.

Value

A list containing per-outcome Erho2 and Phi, optional composite coefficients, universe/relative-error/absolute-error covariance matrices, coefficient scale, allocation, extrapolation flag, and fit diagnostics.

See Also

[gt_score](#), [gt_dstudy](#)

Examples

```
d <- expand.grid(item = 1:12, rater = 1:3)
d$score <- sin(d$item) + d$rater / 5 + cos(d$item * d$rater) / 4
design <- gt_design("item", "rater", random = ~ item + rater)
fit <- gt_fit(d, "score", design,
  control = gt_control(gaussian = list(check_hessian = FALSE, retry_seed = 42)))
gt_reliability(fit)$per_trait
gt_reliability(fit, design = c(rater = 6))$per_trait
```

gt_score

*Declare Composite-Score Weights***Description**

Stores explicit weights without normalizing or choosing a score scale.

Usage

```
gt_score(weights)
```

Arguments

weights	Finite numeric vector with unique outcome names and at least one nonzero entry. Every modeled outcome must be weighted exactly once when used in a reliability calculation.
---------	---

Details

Weights must be substantively meaningful on the chosen outcome scale. A latent binary/ordinal composite is not an observed-score composite. Negative and non-unit-sum weights are allowed.

Value

A gt_score object containing weights and the "as_supplied" normalization policy.

See Also

[gt_reliability](#), [gt_dstudy](#)

Examples

```
gt_score(c(efficacy = 0.5, safety = 0.5))
```

Description

Documents three synthetic task panels: Hate-Speech, Mental-Health, and Drug-Review. Eight outcome sets and fifteen combinations of outcome set and coding are available through [gt_example](#). Every observation is algorithmically simulated; none is an actual LLM response or source-study record.

Format

Calling `gt_example(name)` returns a list. Its data element is a data frame with five shared factor columns and the selected outcomes. Other elements identify outcomes, families, design roles, coding, source, and notes. Installed resources also contain `source_provenance`.

item 24 hypothetical identifiers, `item_01` through `item_24`.

evaluator Three generic identifiers: `evaluator_1`, `evaluator_2`, and `evaluator_3`. These do not identify real models.

prompt Two generic identifiers: `prompt_1` and `prompt_2`.

temp Two factor levels: `temperature_1` and `temperature_2`. These are abstract settings, not actual numeric generation temperatures.

seed Two factor levels: `seed_1` and `seed_2`. No random-number generator was executed with these labels; they identify hypothetical repeats.

Details

Each outcome set has 576 measurement rows: 24 hypothetical items crossed with three generic evaluators, two generic prompts, two temperature-setting identifiers, and two seed identifiers. Each item therefore has 24 measurements. The rows are repeated measurements within a toy panel, not independent real texts. The eight outcome sets are alternative representations of three synthetic panels, not eight independent empirical samples.

The public generator uses deterministic bounded integer arithmetic. It does not read source studies, fit models, draw from an estimated data-generating distribution, call an LLM, or use observed prevalence or performance statistics. These toy patterns demonstrate category handling and data structure. They are not a parameter-recovery experiment or evidence about reliability in a real task.

No raw texts, human labels, clinical measurements, actual model identities, prompt contents, API metadata, or private manuscript observations are included. The task-specific papers and corpus references in the three dedicated help topics motivate the task types only; they supplied none of the simulated rows.

Outcome sets and coding

The [Hate-Speech topic](#) describes one ordinal set. The [Mental-Health topic](#) describes five sets: unordered labels, six binary flags, three binary OR groups, and two Gaussian working-score representations. The [Drug-Review topic](#) describes holistic ordinal sentiment and four ordinal aspects.

`coding = "native"` preserves the declared binary, ordinal, or unordered categories. The Mental-Health 7L/3L variants are explicitly Gaussian working scores under both options, not validated

clinical severity scales. The legacy name `coding = "manuscript"` selects numeric-score compatibility for seven sets. It does not retrieve manuscript observations or reproduce any manuscript findings. The nominal set has native coding only.

Use `gt_example()` to list the catalog. Successful loading does not prove fitting feasibility: random dimensions can exceed the dense discrete backend's limits even in a 576-row panel. Inspect [gt_preflight](#) with the intended family and source design. Reliability scales and numerical acceptance requirements are described in [gt_reliability](#).

Source

Original synthetic data authored by Jin Liu, distributed under GPL-3. The public repository script `'scripts/build_package_data.R'` regenerates the resources using deterministic integer arithmetic. Its `--verify-only` option checks expected objects and the manifest without modifying files. Generator identity is stored in `source_provenance`; resource checksums are recorded in the installed `'extdata/manifest.csv'`.

The installed `'DATASET_REFERENCES.bib'` contains six background references: three task-specific annotation papers and three corpus references. These references are not data provenance for the synthetic observations.

See Also

[gt_example](#), [gtheory_hate_speech](#), [gtheory_mental_health](#), [gtheory_drug_review](#)

Examples

```
gt_example()
x <- gt_example("hate_speech")
dim(x$data)
vapply(x$data[c(x$object, x$facets)], nlevels, integer(1))
x$source_provenance$data_kind
system.file("DATASET_REFERENCES.bib", package = "Gtheory4LLM")
```

gtheory_drug_review *Synthetic Drug-Review Sentiment and Aspect Examples*

Description

Two toy outcome sets for five-category holistic sentiment and four three-category aspects. All observations are synthetic; they are not patient records, real review annotations, actual LLM outputs, or observations from the cited drug-review corpus.

Format

Each call to [gt_example](#) returns a list with a 576-row data frame. Shared factors identify 24 hypothetical items, three generic evaluators, two prompts, two temperature settings, and two seed identifiers. Their levels are documented in the [dataset overview](#).

drug_review One ordered score under native coding: 1 = VERY_NEGATIVE, 2 = NEGATIVE, 3 = NEUTRAL, 4 = POSITIVE, 5 = VERY_POSITIVE. These category names represent holistic sentiment.

drug_review_4aspect Four outcome columns: efficacy, safety, burden, and cost. Each is ordered NEGATIVE, NEUTRAL, POSITIVE under native coding.

Native families are ordinal with probit links. The legacy coding = "manuscript" option returns numerical Gaussian working scores: 1–5 for holistic sentiment and 1–3 for aspects. Other list elements identify outcomes, families, design roles, coding, resource path, notes, and synthetic provenance.

Details

The names illustrate four possible dimensions of review sentiment: treatment effectiveness (efficacy), tolerability (safety), ease of use (burden), and affordability (cost). The toy values were generated by integer arithmetic without reading reviews or evaluating treatments. NEUTRAL is simply the middle simulated category; it does not record an unmentioned aspect, missing response, or patient's stated opinion. Aspects can differ from holistic sentiment.

The two sets share a hypothetical design and are not independent empirical samples. Numeric-score compatibility treats category steps as equally spaced; it retrieves no manuscript data or findings. No source ratings, drug names, clinical outcomes, review text, actual model outputs, prevalence statistics, or fitted parameters are included. Inspect [gt_preflight](#) before fitting the intended source model; a 576-row table can still exceed dense discrete dimension limits.

Source

Original synthetic observations authored by Jin Liu under GPL-3. Public generator:

```
scripts/build_package_data.R
```

The references below motivate the drug-review sentiment task and corpus context. Neither supplied observations for these synthetic tables. See the [dataset overview](#) for the common construction.

References

Gräßer, F., Kallumadi, S., Malberg, H., and Zaunseder, S. (2018). Aspect-Based Sentiment Analysis of Drug Reviews Applying Cross-Domain and Cross-Data Learning. *Proceedings of the 2018 International Conference on Digital Health*, 121–125. doi:10.1145/3194658.3194677.

Liu, J. (2026). Learning to Fuse Aspect-Level LLM Annotations for Low-Quality Ordinal Sentiment Supervision. *SSRN working paper*, 6244519; revised June 17, 2026.

doi:10.2139/ssrn.6244519.

See Also

[Dataset overview](#), [gt_example](#), [gt_family](#)

Examples

```
holistic <- gt_example("drug_review")
str(holistic$data$score)
table(holistic$data$score)
aspects <- gt_example("drug_review_4aspect")
str(aspects$data[aspects$outcomes])
table(aspects$data$efficacy, aspects$data$safety)
```

gtheory_hate_speech *Synthetic Hate-Speech Category Judgments*

Description

An ordinal toy outcome set for illustrating repeated subjective text-category judgments. All 576 observations are synthetic; they are not HateXplain records, crowdworker labels, or actual LLM outputs. Access the table with [gt_example](#).

Format

The returned list contains a data frame with 576 rows and six columns. The shared panel has 24 hypothetical items, three generic evaluators, two prompts, two temperature settings, and two seed identifiers.

item Factor identifying 24 hypothetical items.

evaluator, prompt, temp, seed See the [overview](#) for these factor levels.

score With native coding, an ordered factor with levels 1, 2, 3, representing NORMAL, OFFENSIVE, and HATE. The legacy coding = "manuscript" option returns the same values as numeric Gaussian working scores.

The remaining list elements identify the outcome family, design roles, coding, resource path, interpretation notes, and synthetic provenance.

Details

The native family is ordinal with a probit link. The numeric-score option additionally treats the category steps as equally spaced. It is a compatibility coding of the same synthetic values, not a reproduction of a manuscript study.

The labels were generated by deterministic integer arithmetic without reading texts, consulting annotations, or reproducing source-study prevalence. They illustrate data format and category order only. Their completeness does not describe any real annotation pipeline. A chosen random-source structure must still pass the backend restrictions; use [gt_preflight](#) before fitting.

Source

Original synthetic observations authored by Jin Liu under GPL-3. Public generator:

```
scripts/build_package_data.R
```

See the [dataset overview](#) for the construction and provenance. The references below provide background on rubric-conditioned annotation and the HateXplain task. Neither supplied observations for this synthetic panel.

References

- Mathew, B., Saha, P., Yimam, S. M., Biemann, C., Goyal, P., and Mukherjee, A. (2021). HateXplain: A Benchmark Dataset for Explainable Hate Speech Detection. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(17), 14867–14875. doi:10.1609/aaai.v35i17.17745.
- Liu, J. (2026). Rubric-conditioned large language model labeling: Agreement, uncertainty, and label consistency in subjective text annotation. *Computers in Human Behavior*, 181, 108988. doi:10.1016/j.chb.2026.108988.

See Also

[Dataset overview](#), [gt_example](#), [gt_family](#)

Examples

```
hate <- gt_example("hate_speech")
dim(hate$data)
str(hate$data)
table(hate$data$score)
# Legacy numeric-score option applied to the same synthetic values.
working <- gt_example("hate_speech", coding = "manuscript")
str(working$data$score)
```

gtheory_mental_health *Synthetic Mental-Health Category and Flag Examples*

Description

Five toy outcome sets illustrating unordered categories, binary flags, binary OR groups, and two numerical working-score mappings. All values are synthetic. They are not clinical data, real text annotations, actual LLM responses, or observations from the cited mental-health corpus.

Format

Each call to [gt_example](#) returns a list with a 576-row data frame. Its shared factors identify 24 hypothetical items, three generic evaluators, two prompts, two temperature settings, and two seeds. See the [dataset overview](#) for their levels. Additional columns depend on the outcome set:

mental_health_nominal One unordered label factor with levels NORMAL, STRESS, ANXIETY, DEPRESSION, BIPOLAR, PERSONALITY_DISORDER, SUICIDAL. The categorical family uses softmax with NORMAL as reference. Native coding only.

mental_health_6flag Six integer 0/1 toy flags:

- depression
- anxiety
- suicidal
- stress
- bipolar
- personality_disorder

Native families are binary with probit links. A one is a simulated flag, not evidence of symptoms or a diagnosis.

mental_health_3group Three integer 0/1 columns:

- stress: the synthetic stress flag.
- anxiety_depression: anxiety OR depression.
- bipolar_personality_suicidal: bipolar OR personality-disorder OR suicidal flags.

These groups may co-occur; they are not mutually exclusive categories. Native families are binary with probit links.

mental_health_7L One integer score: NORMAL = 1, STRESS = 2, ANXIETY = 3, DEPRESSION = 4, BIPOLAR = 5, PERSONALITY_DISORDER = 6, SUICIDAL = 7. It is a Gaussian working score under both coding options.

mental_health_3L One integer score: NORMAL or STRESS = 1; ANXIETY or DEPRESSION = 2; BIPOLAR, PERSONALITY_DISORDER, or SUICIDAL = 3. It is a Gaussian working score under both coding options.

Other list elements identify outcomes, families, design roles, coding, resource path, notes, and synthetic provenance.

Details

The 7L/3L numerical mappings do not establish clinical severity order or equal distances between categories. The nominal set preserves unordered identity. The six binary flags are not diagnoses. The three-group binary set differs from the single-outcome 3L working score.

For flags and groups, the legacy `coding = "manuscript"` option keeps 0/1 values but declares Gaussian working-score families. The 7L/3L sets are Gaussian even under native coding. This compatibility argument applies to synthetic values only and reproduces no manuscript observations or results.

The five representations share one hypothetical design. The nominal label determines the 7L/3L mappings; the six generated flags determine the OR groups. No observations, prevalence estimates, fitted parameters, texts, or actual annotations were copied from any source study. These deterministic toy patterns are not a parameter-recovery simulation. See [gt_preflight](#) for structural and resource checks before fitting.

Source

Original synthetic observations authored by Jin Liu under GPL-3. Public generator:

`scripts/build_package_data.R`

The references below motivate the affective-text annotation task and its corpus context. Neither supplied observations for these synthetic tables. The common construction is described in the [dataset overview](#).

References

Sarkar, S. (n.d.). *Sentiment Analysis for Mental Health*. Kaggle dataset; accessed September 12, 2026. [Dataset record on Kaggle](#).

Liu, J. (2025). LLM-Based Annotation as Multi-View Supervision for Affective Text Classification: Soft Labels, Multi-Task Decomposition, and Entropy Diagnostics. *SSRN working paper*, 5964738; revised March 19, 2026. [doi:10.2139/ssrn.5964738](#).

See Also

[Dataset overview](#), [gt_example](#), [gt_family](#), [gt_score](#)

Examples

```
nominal <- gt_example("mental_health_nominal")
str(nominal$data$label)
table(nominal$data$label)
flags <- gt_example("mental_health_6flag")
str(flags$data[flags$outcomes])
groups <- gt_example("mental_health_3group")
str(groups$data[groups$outcomes])
working <- gt_example("mental_health_3L")
table(working$data$score)
```

print.summary.gt_fit *Print a Concise Generalizability Model Summary*

Description

Displays model scope, numerical acceptance, likelihood, source variances, and fixed estimates without printing the fitted backend model or observation data.

Usage

```
## S3 method for class 'summary.gt_fit'
print(x, ..., digits = max(3L,
  getOption("digits") - 3L))
```

Arguments

x	A summary.gt_fit object returned by summary(fit).
...	Additional arguments; currently unused.
digits	Number of significant digits for printed estimates, from 1 to 22.

Details

The printed summary reports the selected attempt, recorded failed attempts, acceptance failures, natural covariance boundaries, artificial parameter bounds, and likelihood-approximation status separately. If the native Gaussian transcript does not identify a unique selected trial, the output says “not identified”. An accepted fit can have earlier failed attempts or a natural covariance boundary. A rejected fit’s estimates are printed for diagnosis only; it remains ineligible for reliability or decision-study calculations.

The printed variance table shows at most 20 rows and the attempt-failure table at most 6 rows. Complete tables and covariance matrices remain in the summary.

Use gt_diagnostics(fit) for full diagnostic details.

Fixed location or contrast estimates are printed on the fitted family’s scale. Numerical acceptance and first-order Laplace approximation adequacy remain separate; the summary makes no additional claim of statistical validation.

Value

Returns x invisibly. The summary object remains a list with outcome and family specifications, source covariance matrices, a source-variance table, means or coefficients, thresholds, likelihood, and diagnostics.

See Also

[gt_fit](#), [gt_diagnostics](#), [gt_components](#)

Examples

```
d <- expand.grid(item = 1:12, rater = 1:3)
d$score <- sin(d$item) + d$rater / 5 + cos(d$item * d$rater) / 4
design <- gt_design("item", "rater", random = ~ item + rater)
fit <- gt_fit(d, "score", design,
  control = gt_control(gaussian = list(check_hessian = FALSE, retry_seed = 42)))
s <- summary(fit)
print(s, digits = 3)
s$variances
```

Index

* datasets

- gtheory_datasets, [19](#)
- gtheory_drug_review, [20](#)
- gtheory_hate_speech, [22](#)
- gtheory_mental_health, [23](#)

- Dataset overview, [21](#), [23](#), [24](#)
- dataset overview, [3](#), [20–24](#)
- Drug-Review, [3](#)
- Drug-Review topic, [19](#)
- drug_review (gtheory_drug_review), [20](#)
- drug_review_4aspect
(gtheory_drug_review), [20](#)

- glm, [3](#)
- gt_components, [3](#), [25](#)
- gt_control, [4](#), [9](#), [15](#)
- gt_design, [3](#), [7](#), [15](#)
- gt_diagnostics, [4](#), [7](#), [8](#), [15](#), [16](#), [25](#)
- gt_dstudy, [3](#), [10](#), [16–18](#)
- gt_example, [3](#), [11](#), [19–24](#)
- gt_family, [8](#), [12](#), [12](#), [15](#), [21](#), [23](#), [24](#)
- gt_fit, [3](#), [4](#), [7–9](#), [12](#), [13](#), [13](#), [15](#), [16](#), [25](#)
- gt_preflight, [3](#), [12](#), [15](#), [20–22](#), [24](#)
- gt_reliability, [10](#), [16](#), [17](#), [18](#), [20](#)
- gt_score, [10](#), [17](#), [18](#), [24](#)
- Gtheory4LLM (Gtheory4LLM-package), [2](#)
- Gtheory4LLM-datasets
(gtheory_datasets), [19](#)
- Gtheory4LLM-package, [2](#)
- gtheory_datasets, [12](#), [19](#)
- gtheory_drug_review, [12](#), [20](#), [20](#)
- gtheory_hate_speech, [12](#), [20](#), [22](#)
- gtheory_mental_health, [12](#), [20](#), [23](#)

- Hate-Speech, [3](#)
- Hate-Speech topic, [19](#)
- hate_speech (gtheory_hate_speech), [22](#)

- Mental-Health, [3](#)
- Mental-Health topic, [19](#)
- mental_health_3group
(gtheory_mental_health), [23](#)
- mental_health_3L
(gtheory_mental_health), [23](#)

- mental_health_6flag
(gtheory_mental_health), [23](#)
- mental_health_7L
(gtheory_mental_health), [23](#)
- mental_health_nominal
(gtheory_mental_health), [23](#)
- overview, [22](#)
- plot.gt_dstudy (gt_dstudy), [10](#)
- print method, [14](#)
- print.gt_fit (gt_fit), [13](#)
- print.gt_preflight (gt_preflight), [15](#)
- print.summary.gt_fit, [25](#)
- summary.gt_fit (gt_fit), [13](#)